

PathFinder.java

```
1  package inf112.peripheryplanet.map;
2
3  import java.util.List;
4  import java.util.Map;
5
6  import com.badlogic.gdx.math.Polygon;
7  import com.badlogic.gdx.math.Vector2;
8
9  public interface PathFinder {
10
11      /** create a new path finder */
12      static PathFinder finder(Map<Vector2, MapCell> map) {
13          return new PathFinderBaseImpl(map);
14      }
15
16      /** where to start from */
17      PathFinder from(Vector2 pos);
18
19      /** where we want to go to */
20      PathFinder to(Vector2 pos);
21
22      /**
23       * a waypoint we want to include in the path (can be given multiple times)
24       */
25      PathFinder via(Vector2 pos);
26
27      /**
28       * a (dangerous?) area we should avoid (can be given multiple times)
29       */
30      PathFinder avoid(Polygon area);
31
32      /** prefer the shortest path */
33      PathFinder shortest();
34
35      /** prefer the safest path */
36      PathFinder safest();
37
38      /** prefer the fastest path */
39      PathFinder fastest();
40
41      /**
42       * calculate and return the path (null if destination is unreachable)
43       */
44      List<Vector2> calculate();
45  }
```

PathFinderImpl.java

```
1  package inf112.peripheryplanet.map;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.Map;
6
7  import com.badlogic.gdx.math.Vector2;
8
9  import inf112.peripheryplanet.pawns.Bird;
10 import inf112.peripheryplanet.pawns.Cat;
11 import inf112.peripheryplanet.pawns.Dodo;
12 import inf112.peripheryplanet.pawns.Duck;
13 import inf112.peripheryplanet.pawns.Pawn;
14
15 // pathfinder algorithm implemented here
16 public class PathFinderImpl extends PathFinderBaseImpl implements PathFinder {
17     public PathFinderImpl(Map<Vector2, MapCell> map) {
18         super(map);
19     }
20
21     // OLD: pathfinder will prefer cells with *low* cost
22     protected double calculateCostForMapCell(MapCell mapCell) {
23         if(mapCell == null)
24             return 1; // default cost
25         else if(mapCell.pawn() != null)
26             return Double.POSITIVE_INFINITY; // already occupied
27         else
28             return mapCell.terrain().movementCost();
29     }
30
31     // NEW: pathfinder will prefer cells with *low* cost
32     protected double calculateCostForMapCell(MapCell mapCell, Pawn pawn) {
33         if(mapCell == null)
34             return 1; // default cost
35         else if(pawn instanceof Bird && !(pawn instanceof Dodo))
36             return 1; // birds (except dodos) can fly, ignore terrain
37         else if(mapCell.pawn() != null && !(pawn instanceof Cat)) // cats can sneak past other pawns
38             return Double.POSITIVE_INFINITY; // already occupied
39         else if(mapCell.terrain() instanceof Water && pawn instanceof Duck) // can swim
40             return 1;
41         else if(pawn.footwear() instanceof RollerSkates && mapCell.terrain() instanceof Paved)
42             // roller skates are extra fast on paved terrain
43             return mapCell.terrain().movementCost() / 2;
44         else
45             return mapCell.terrain().movementCost();
46     }
47
48     protected List<Vector2> calculatePath() {
49         List<Vector2> result = new ArrayList<>();
50         // ...
51         return result;
52     }
53 }
54 }
```

MapCell.java

```
1 package inf112.peripheryplanet.map;
2
3 import java.util.List;
4
5 import inf112.peripheryplanet.pawns.Pawn;
6 import inf112.peripheryplanet.terrain.Terrain;
7
8 public interface MapCell {
9     /** terrain at this location (determines movement cost) */
10     Terrain terrain();
11
12     /**
13      * the pawn (colonist, animal, etc) currently at this location (only one
14      * allowed!)
15      */
16     Pawn pawn();
17
18     /**
19      * replace the pawn (colonist, animal, etc) currently at this location (only one
20      * allowed!)
21      */
22     Pawn pawn(Pawn newPawn);
23
24     /** other items at this location (latest added first) */
25     List<MapElement> items();
26
27 }
```